



UNIVERSITÀ DI PARMA

Pointers to Pointers

*Bad programmers worry about the code.
Good programmers worry about data
structures and their relationships*

Linus Torvalds

- Pointer to Pointers
- Dynamic allocation of multidimensional arrays
- Arrays of Strings

SUMMARY





UNIVERSITÀ DI PARMA

Pointers to Pointers

- Also pointer variables feature a memory address
- Can we store that address?
- Yes → pointers to pointers
- You need to use a double `**` in definitions
- i.e. `int **p2p; //p2p is a double pointer`
- Also pointer to pointer variables feature a memory address...
 - You can use a lot of `'*'`...



Pointers to Pointers: syntax

```
int var;
```

```
var = 10;
```

```
int *ptr;
```

```
ptr = &var;
```

```
*ptr = 33;
```

```
int **dptr;
```

```
dptr = &ptr;
```

```
**dptr = 57; // note the double **
```

0x7fff387eaea4

57

0x7fff387eaea8

0x7fff387eaea4

0x7fff387eaeb0

0x7fff387eaea8



- Do you remember the main() function?
 - `int main(int argv, char **argc);`
- Why we need them?
 - When dealing with functions that have side effects, or
 - In order to dynamically allocate blocks of memory with more than one dimension (e.g. matrices)
 - Often when managing multidimensional arrays





UNIVERSITÀ DI PARMA

Dynamic allocation of multidimensional arrays

- For sake of simplicity, just consider a bi-dimensional array
- A statically allocated 2D array can be defined as:

```
float b[33][55];
```

- To access a specific element, we can use a syntax like:

```
b[17][34] = ... // we MUST use [][] and indexes
```

- To access array address

```
printf("the address is %p\n", b) // we have to omit  
[]
```


- Can we use a single index with a bi-dimensional array?
 - i.e., is it correct to use and what is the meaning of
`b[17]` ??
- When we omit some dimension, this always represents an address. Which one?
- We can see `b` as an **array of arrays**, namely `b[17]` is the address of the 18th row
`printf("the address of 18th row is %p\n", b[17])`

- 6 different possibilities:
 - 1) **1D array with pointer arithmetic**
 - 2) *VLA of pointers*
 - 3) **Double pointer with a malloc() for each row**
 - 4) *Double pointer and a single malloc()*
 - 5) *A pointer to a VLA*
 - 6) *A pointer to the first row of a VLA*
- We are going to discuss #1 and #3 only

1D array with pointer arithmetic

- When we want to manage a matrix with ***r* rows** and ***c* columns**
- Simply allocate a block of ***r*c* elements**, i.e.,

```
float *matr = malloc(r*c*sizeof(float));
```
- The element in row ***y*** and column ***x*** can be easily accessed as

```
matr[x + y*c] = ...
```
- Pros → easy allocation (1 malloc())
- Cons → we need to use a different syntax for access + math

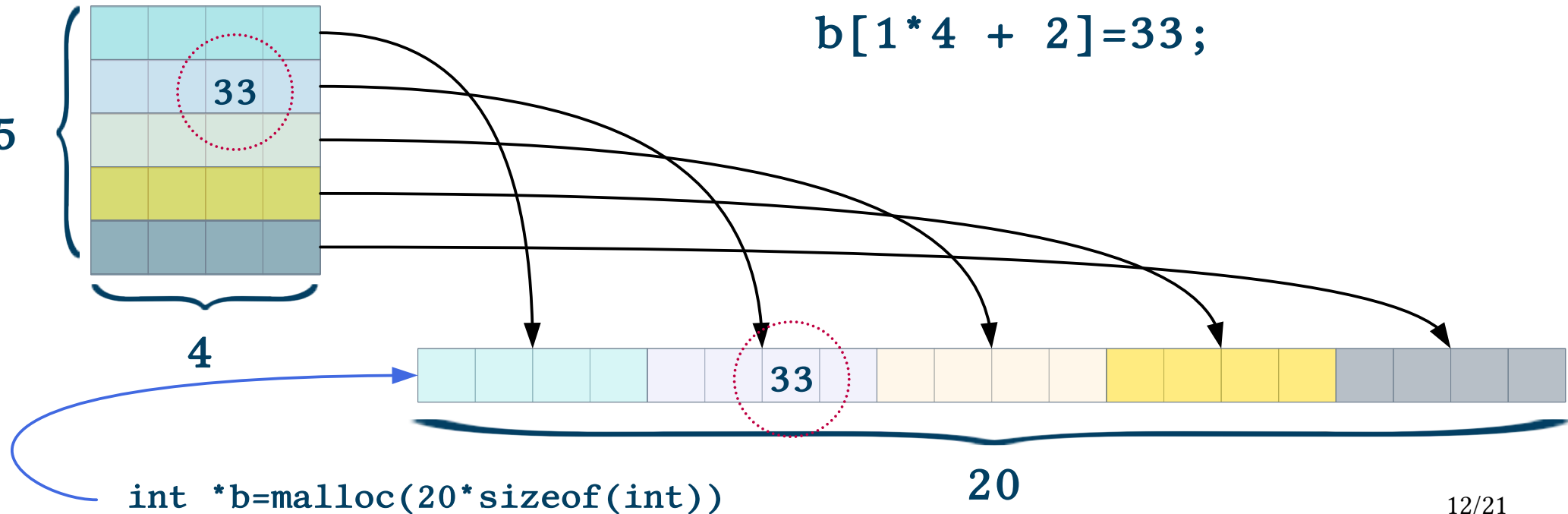
1D array with pointer arithmetic

- Logical vs “Physical” layout

`a[1][2]=33;`

`int a[5][4]`

`b[1*4 + 2]=33;`



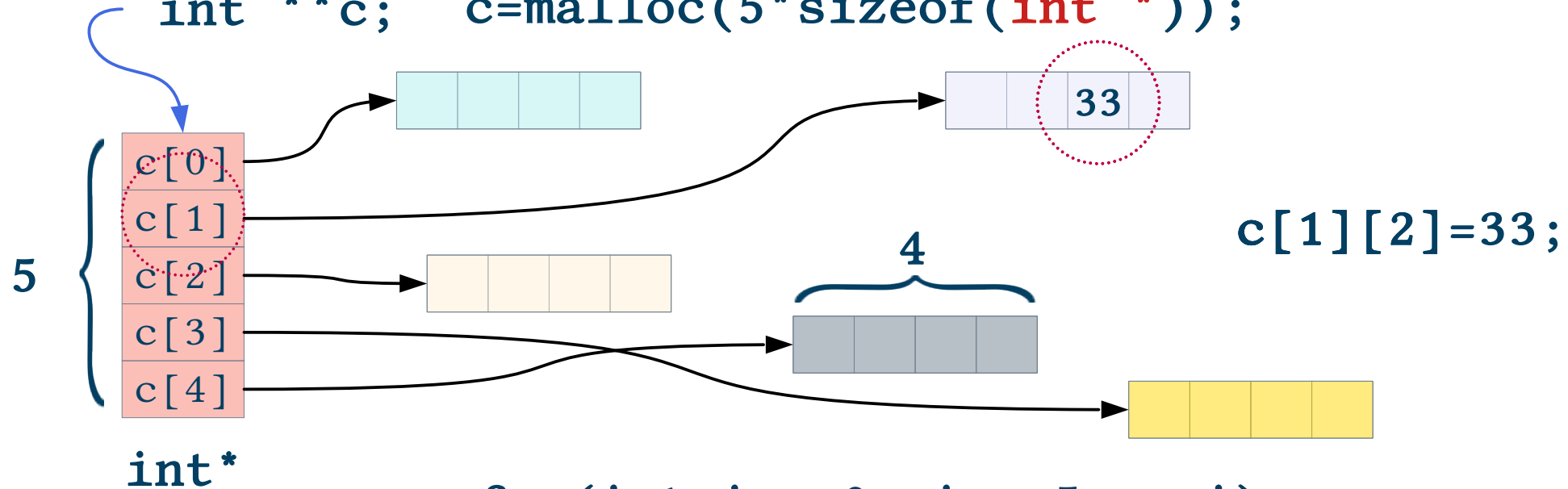
Double pointer with a malloc() for each row

- When we want to manage a matrix with ***r* rows** and ***c* columns**
- A double pointer can be used to allocate *r* “row” pointers, one for each row
- Each row is then independently allocated
- Pros:
 - Same syntax as multidim arrays, i.e., `[][]`
 - Can also manage different sized rows
- Cons: complex memory allocation & destroy

Double pointer with a malloc() for each row

- As before we want to dynamically allocate a 5×4 matrix

```
int **c;  c=malloc(5*sizeof(int *));
```



```
for(int i = 0; i < 5; ++i)  
    c[i] = malloc(4*sizeof(int));
```



UNIVERSITÀ DI PARMA

Arrays of strings

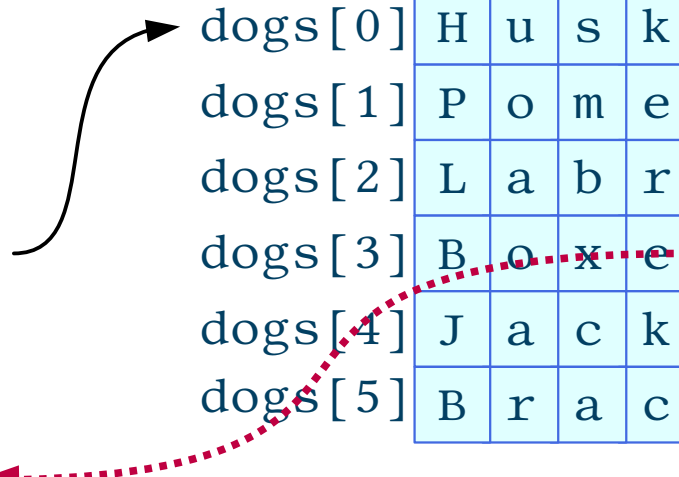
- La stringhe già sono memorizzate in array
 - Posso usare array di array
- Array bidimensionali
 - Devo dimensionare ogni riga con la lunghezza massima prevista
 - Spreco spazio
- Array di puntatori + allocazione dinamica
 - Più complesso ma ottimizzo uso spazio

- Recap: for a bi-dimensional array i.e. `char myarr[n][m]`
 - `myarr[x][y]` → single char at (x, y) position
 - `myarr[r]` → r^{th} row address

`char dogs[6][15]`

<code>dogs[0]</code>	H	u	s	k	y	\0	?	?	?	?	?	?	?	?
<code>dogs[1]</code>	P	o	m	e	r	a	n	i	a	\0	?	?	?	?
<code>dogs[2]</code>	L	a	b	r	a	d	o	r	\0	?	?	?	?	?
<code>dogs[3]</code>	B	o	x	e	r	\0	?	?	?	?	?	?	?	?
<code>dogs[4]</code>	J	a	c	k		R	u	s	s	e	l	\0	?	?
<code>dogs[5]</code>	B	r	a	c	c	h	e	t	t	o	\0	?	?	?

`dogs[3][4]`

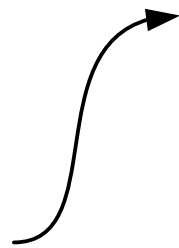


- Therefore, using a single index $\rightarrow$ address of a “string”
- This can be easily used for all functions that involve strings
- As examples:

```
printf(“The dog in the 4th row is %s\n”, dogs[3]);  
scanf(“%[^\n]”, dogs[5]);  
strcpy(dogs[0], dogs[3]);  
...
```

- Use of bi-dimensional arrays
 - Pro: somewhat easier than alternatives
 - Cons: not memory efficient or flexible

char dogs[6][15]

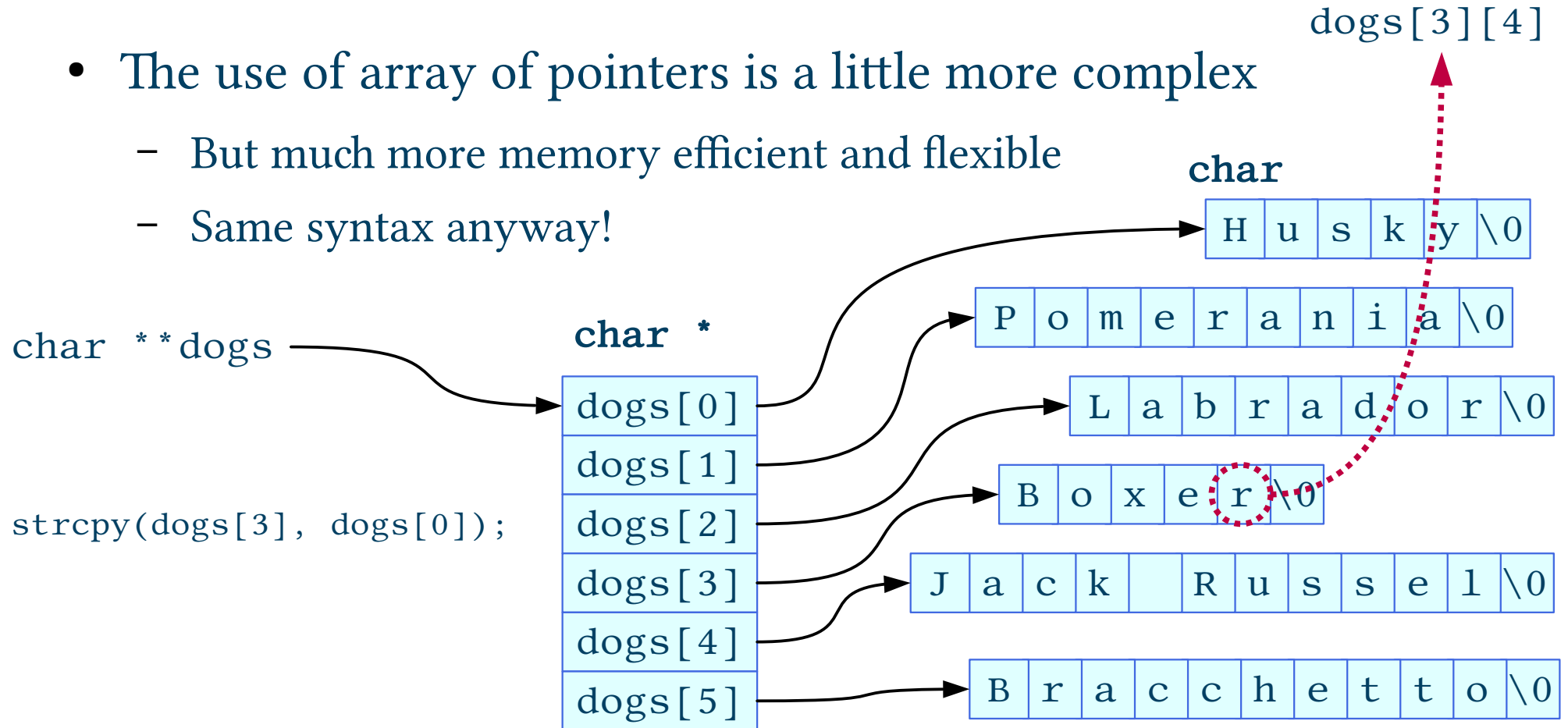


dogs[0]	H	u	s	k	y	\0	?	?	?	?	?	?	?	?
dogs[1]	P	o	m	e	r	a	n	i	a	\0	?	?	?	?
dogs[2]	L	a	b	r	a	d	o	r	\0	?	?	?	?	?
dogs[3]	B	o	x	e	r	\0	?	?	?	?	?	?	?	?
dogs[4]	J	a	c	k		R	u	s	s	e	l	\0	?	?
dogs[5]	B	r	a	c	c	h	e	t	t	o	\0	?	?	?

In this example
around 40% space wasted

Arrays of strings

- The use of array of pointers is a little more complex
 - But much more memory efficient and flexible
 - Same syntax anyway!





UNIVERSITÀ DI PARMA

Memory Allocation & Pointers



KEEP
CALM
IT'S
QUESTION
TIME

Do we pay for bytes? No, so what...

Z, VisLab srl, 2014

- Pointer to Pointers
- Dynamic allocation of multidimensional arrays
- Arrays of Strings

SUMMARY





- Also pointer variables feature a memory address
- Can we store that address?
- Yes → pointers to pointers
- You need to use a double `**` in definitions
- i.e. `int **p2p; //p2p is a double pointer`
- Also pointer to pointer variables feature a memory address...
 - You can use a lot of `*`...



Pointers to Pointers: syntax



```
int var;
var = 10;

int *ptr;
ptr = &var;
*ptr = 33;

int **dptr;
dptr = &ptr;
**dptr = 57; // note the double **
```

0x7fff387eaea4

57

0x7fff387eaea8

0x7fff387eaea4

0x7fff387eae00

0x7fff387eaea8





- Do you remember the main() function?
 - `int main(int argv, char **argc);`
- Why we need them?
 - When dealing with functions that have side effects, or
 - In order to dynamically allocate blocks of memory with more than one dimension (e.g. matrices)
 - Often when managing multidimensional arrays





- For sake of simplicity, just consider a bi-dimensional array
- A statically allocated 2D array can be defined as:
`float b[33][55];`
- To access a specific element, we can use a syntax like:
`b[17][34] = ... // we MUST use [][] and indexes`
- To access array address
`printf("the address is %p\n", b) // we have to omit []`



- Can we use a single index with a bi-dimensional array?
 - i.e., is it correct to use and what is the meaning of
`b[17]` ??
- When we omit some dimension, this always represents an address. Which one?
- We can see `b` as an **array of arrays**, namely `b[17]` is the address of the 18th row
`printf("the address of 18th row is %p\n", b[17])`



- 6 different possibilities:
 - 1) **1D array with pointer arithmetic**
 - 2) *VLA of pointers*
 - 3) **Double pointer with a malloc() for each row**
 - 4) *Double pointer and a single malloc()*
 - 5) *A pointer to a VLA*
 - 6) *A pointer to the first row of a VLA*
- We are going to discuss #1 and #3 only



- When we want to manage a matrix with ***r* rows** and ***c* columns**
- Simply allocate a block of ***r*c* elements**, i.e.,
`float *matr = malloc(r*c*sizeof(float));`
- The element in row ***y*** and column ***x*** can be easily accessed as
`matr[x + y*c] = ...`
- Pros → easy allocation (1 malloc())
- Cons → we need to use a different syntax for access + math

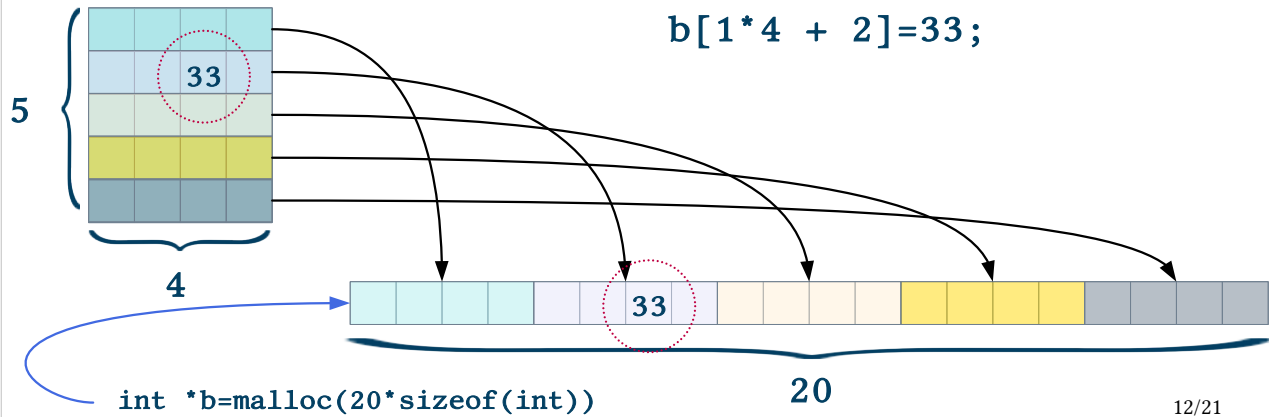
1D array with pointer arithmetic

- Logical vs “Physical” layout

`a[1][2]=33;`

`int a[5][4]`

`b[1*4 + 2]=33;`



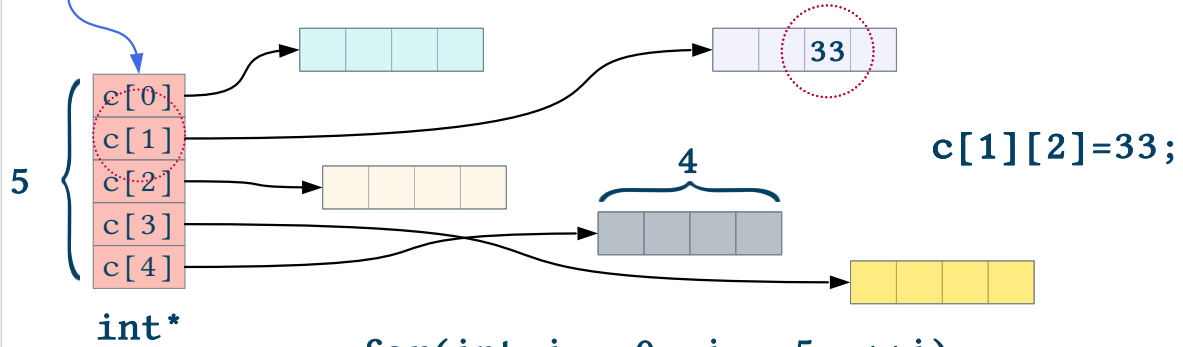


- When we want to manage a matrix with ***r* rows** and ***c* columns**
- A double pointer can be used to allocate *r* “row” pointers, one for each row
- Each row is then independently allocated
- Pros:
 - Same syntax as multidim arrays, i.e., `[][]`
 - Can also manage different sized rows
- Cons: complex memory allocation & destroy

Double pointer with a malloc() for each row

- As before we want to dynamically allocate a 5×4 matrix

```
int **c; c=malloc(5*sizeof(int *));
```



```
for(int i = 0; i < 5; ++i)  
    c[i] = malloc(4*sizeof(int));
```




- Le stringhe già sono memorizzate in array
 - Posso usare array di array
- Array bidimensionali
 - Devo dimensionare ogni riga con la lunghezza massima prevista
 - Spreco spazio
- Array di puntatori + allocazione dinamica
 - Più complesso ma ottimizzo uso spazio

- Recap: for a bi-dimensional array i.e. `char myarr[n][m]`

- `myarr[x][y]` → single char at (x, y) position
- `myarr[r]` → r^{th} row address

`char dogs[6][15]`

<code>dogs[0]</code>	H	u	s	k	y	\0	?	?	?	?	?	?	?	?	?
<code>dogs[1]</code>	P	o	m	e	r	a	n	i	a	\0	?	?	?	?	?
<code>dogs[2]</code>	L	a	b	r	a	d	o	r	\0	?	?	?	?	?	?
<code>dogs[3]</code>	B	o	x	e	r	\0	?	?	?	?	?	?	?	?	?
<code>dogs[4]</code>	J	a	c	k		R	u	s	s	e	l	\0	?	?	?
<code>dogs[5]</code>	B	r	a	c	c	h	e	t	t	o	\0	?	?	?	?

`dogs[3][4]`



- Therefore, using a single index $\rightarrow$ address of a “string”
- This can be easily used for all functions that involve strings
- As examples:

```
printf("The dog in the 4th row is %s\n", dogs[3]);  
scanf("%[^\n]", dogs[5]);  
strcpy(dogs[0], dogs[3]);  
...
```

- Use of bi-dimensional arrays
 - Pro: somewhat easier than alternatives
 - Cons: not memory efficient or flexible

char dogs[6][15]

In this example
around 40% space wasted



dogs[0]	H	u	s	k	y	\0	?	?	?	?	?	?	?	?	?
dogs[1]	P	o	m	e	r	a	n	i	a	\0	?	?	?	?	?
dogs[2]	L	a	b	r	a	d	o	r	\0	?	?	?	?	?	?
dogs[3]	B	o	x	e	r	\0	?	?	?	?	?	?	?	?	?
dogs[4]	J	a	c	k		R	u	s	s	e	l	\0	?	?	?
dogs[5]	B	r	a	c	c	h	e	t	t	o	\0	?	?	?	?



- The use of array of pointers is a little more complex
 - But much more memory efficient and flexible
 - Same syntax anyway!

`char **dogs`

`strcpy(dogs[3], dogs[0]);`

`char *`

`dogs[0]`

`dogs[1]`

`dogs[2]`

`dogs[3]`

`dogs[4]`

`dogs[5]`

`char`

H u s k y \0

P o m e r a n i a \0

L a b r a d o r \0

B o x e r \0

J a c k R u s s e l \0

B r a c c h e t t o \0

`dogs[3][4]`



